Submission 2

Integrated Navigation for Aircraft:
Fault Detection and Multi-Objective Design Optimisation

0 Introduction

Submission 1 produced an 18-state EKF that estimates time-varying additive faults via a random walk model. This submission converts the fault estimates from that into binary detection events using a two-sided CUSUM detector (Task 4), then optimises the entire fault-detection design as a multi-objective optimisation problem over the CUSUM thresholds and the random-walk noise covariance (Task 5). The goal is to move from a hand-tuned design to one that is justified by cost functions and trade-offs between false alarms, detection delay, and state-estimation quality.

1 Fault Detection with CUSUM

1.1 CUSUM Parameter Selection

The two-sided CUSUM operates on a normalised fault residual $s_k = (\hat{b}_k - \mu_0)/\sigma_0$, where μ_0 and σ_0 are the pre-fault mean and standard deviation of each estimated fault channel. Pre-fault statistics are extracted from `dataTask2`, which contains constant biases but no time-varying faults, after the filter has converged (last 70% of the run). The recursions

$$S_k^+ = \max\big(0,\ S_{k-1}^+ + s_k - \gamma\big), \qquad S_k^- = \max\big(0,\ S_{k-1}^- - s_k - \gamma\big), \tag{1}$$

raise an alarm when either S_k^+ or S_k^- exceeds the threshold δ. After every alarm both accumulators are reset and a 5 s wait window is enforced so that a single constant fault doesn't keep triggering repeatedly; instead it set one alarm event per wait window for as long as the fault is still present.

A single set of parameters is shared across all six IMU channels, with σ_0 providing normalisation for each of the channels. The leakage γ must exceed the typical magnitude of $|s_k|$ under nominal operation (otherwise the accumulator will drift upwards from the noise); for $s_k \sim \mathcal{N}(0,1)$ this means $\gamma > 1$. The threshold δ is then chosen large enough that `dataTask2` calls no false alarms over the 'post warm up' window. A few manual iterations produce $\gamma = 2.5$ and $\delta = 15$, giving zero false alarms on `dataTask2` and a mean detection delay of 1.78 s on `dataTask3`.

1.2 Detection Results

Figure 1 shows the six estimated fault channels from the random walk EKF on `dataTask3`, with each red dashed line marking a CUSUM alarm. The figures show that the alarm is clustering successfully where the faults occur. The repeated alerts work successfully as seen where the red lines pause the CUSUM detection as long as the fault continues to drive $|s_k|$ past γ; this ensures the supervisor is aware of a persistent issue without being overwhelmed by constant alarms. Channel b_q produces no alarm on this trace, consistent with the absence of any large s_k leaps in that estimated fault. No alarms are raised on `dataTask2`, confirming that the chosen (γ, δ) pair separates genuine faults from nominal random-walk drift.

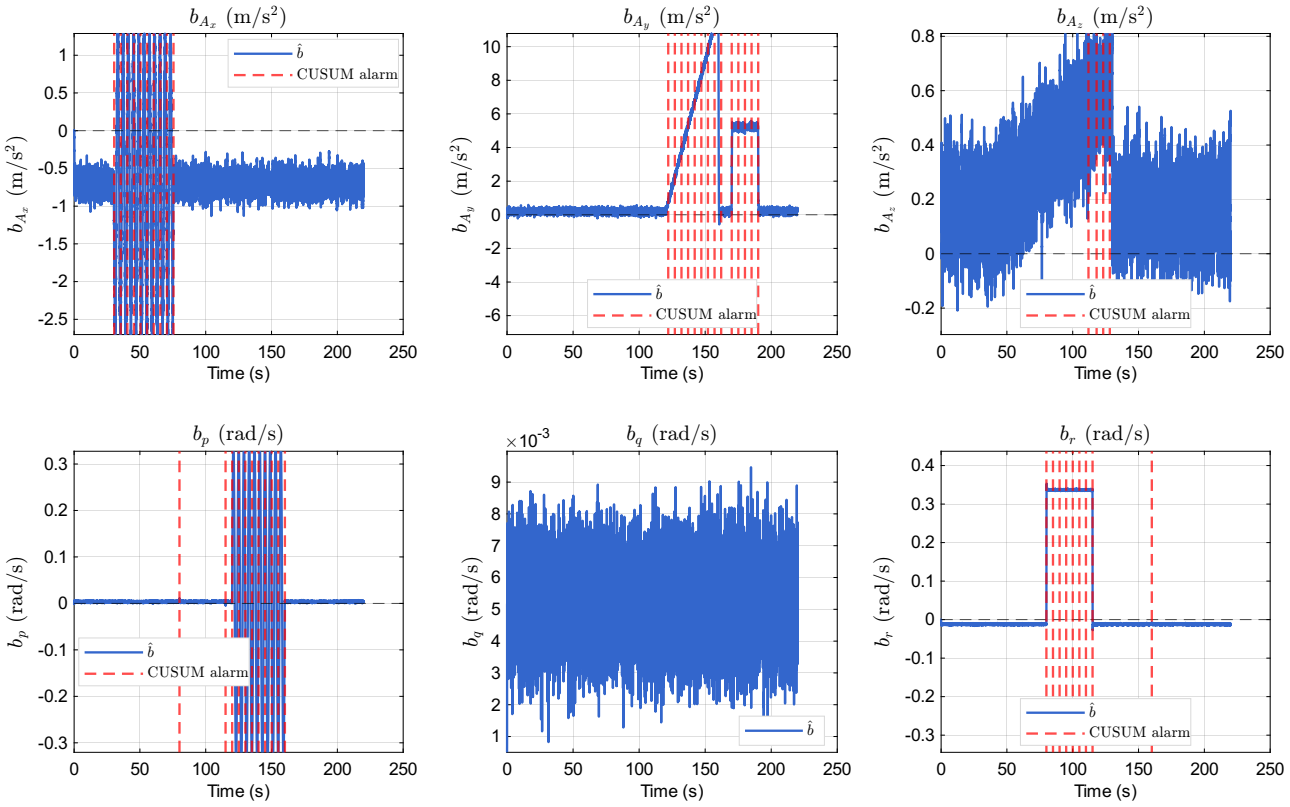

Figure 1: Estimated fault signals on `dataTask3` (blue) with CUSUM alarms (red dashed). Axes are scaled to the post-convergence range so that fault transitions and the alarm timing are clearly visible. The refractory window of 5 s produces one alarm event per window during a sustained fault. Shared parameters $\gamma = 2.5$, $\delta = 15$ across all six channels.

2 Fault Detection Design Optimisation

2.1 Performance Criteria and Trade-offs

Three objectives to be minimised capture the engineering trade-off:

- J_1 – **false alarm count** on `dataTask2`: number of CUSUM events raised on data containing only constant biases. This controls detector specificity.
- J_2 – **mean detection delay** on `dataTask3`: mean time between each labelled fault onset and the next alarm in the same channel, capped at 30 s if no alarm occurs. Reference onsets are obtained from a high-resolution oracle pass over the unconstrained $\hat{b}$. This controls detector responsiveness.
- J_3 – **innovation RMS** on the GPS velocity and attitude channels of `dataTask3`: a state-quality proxy that penalises designs where an aggressive random-walk noise inflates the filter's error.

These three criteria play into one another: tightening δ shrinks J_1 but inflates J_2, and increasing σ_b shrinks J_2 but inflates J_3 because the EKF starts to track measurement noise into the fault state. J_2 vs. J_1 is the standard trade-off; J_3 couples the detector design back into the estimator and is what makes this a true multi-objective problem rather than a pure CUSUM-tuning exercise.

2.2 Safety and Sustainability

In an integrated navigation system, undetected faults propagate through the rotation matrix into body-velocity estimates and ultimately into the flight control loop. A miss on J_2 therefore directly threatens flight safety, especially for dynamic manoeuvres where the observation model is very nonlinear. In contrast, a high J_1 creates the opposite hazard: nuisance alarms desensitise the crew or the autopilot's reconfiguration logic and cause unnecessary fall-back to redundant sensors, which is a major contributor to loss of control incidents.

From a sustainability perspective, a non-optimal design will produce costs over the certification and entire maintenance lifecycle. Designs that miss real faults force expensive flight-test reruns and additional redundancy, increasing aircraft weight and lifetime fuel burn. Designs that over-trigger waste maintenance hours on inspections where no faults are found. A multi-objective formulation makes the trade-off explicit and auditable rather than embedding it in a single value, which leaves ambiguity between the designer and the certification authority.

2.3 Optimisation Study

Summary

A four-variable, three-objective optimisation over the CUSUM parameters (γ, δ) and the random-walk noise standard deviations $(\sigma_{b,a}, \sigma_{b,g})$ produces a Pareto front of three non-dominated designs from a 2304 sample search. The knee-point design (Table 1) cuts mean detection delay from 1.78 s to 0.37 s at no cost in false alarms ($J_1 = 0$ in both) and with essentially identical J_3.

Problem formulation

Decision variables are taken in $\log_{10}$ space to span the relevant orders of magnitude with uniform sampling. The design vector is:

$$\mathbf{x} = [\log_{10}\gamma,\ \log_{10}\delta,\ \log_{10}\sigma_{b,a},\ \log_{10}\sigma_{b,g}]^\top, \qquad \mathbf{x} \in [\mathbf{l}, \mathbf{u}],$$

with bounds $\mathbf{l} = [-1.5,\ 0,\ -2.5,\ -3.5]$ and $\mathbf{u} = [0.7,\ 1.7,\ -0.7,\ -1.5]$. "Shared" is taken to mean shared across all six channels for (γ, δ) and shared within each sensor type for σ_b, since accelerometer and gyroscope process noises have incompatible units. Objectives $\mathbf{J}(\mathbf{x}) = [J_1, J_2, J_3]^\top$ are evaluated by re-running the random walk EKF and CUSUM detector on `dataTask2` and `dataTask3`; the optimisation problem is $\min_{\mathbf{x}\in[\mathbf{l},\mathbf{u}]} \mathbf{J}(\mathbf{x})$ in the Pareto sense. No equality or inequality constraints are imposed beyond the box boundaries.

Sampling plan

Three plans are compared on the four-dimensional unit cube: Latin Hypercube Sampling (LHS), a 3^4 Full Factorial truncated to 64 points, and a scrambled Sobol low-discrepancy sequence. Figure 2 shows the projection onto all six pairs of design variables. The Full Factorial achieves the largest minimum pairwise distance (0.500) but only because it is restricted to three values per axis, leaving every parameter combination in between unexplored which makes it a poor candidate. The scrambled Sobol sequence (0.157) and LHS (0.146) deliver more samples, with Sobol producing slightly more uniform pair projections and LHS guaranteeing uniform marginal coverage. For this study a different sampling method, a structured grid in $(\sigma_{b,a}, \sigma_{b,g})$, is chosen as the actual sampling plan because it lets each EKF run be cached and re-used across all (γ, δ) candidates. This give faster compute time and good sample space resolution.

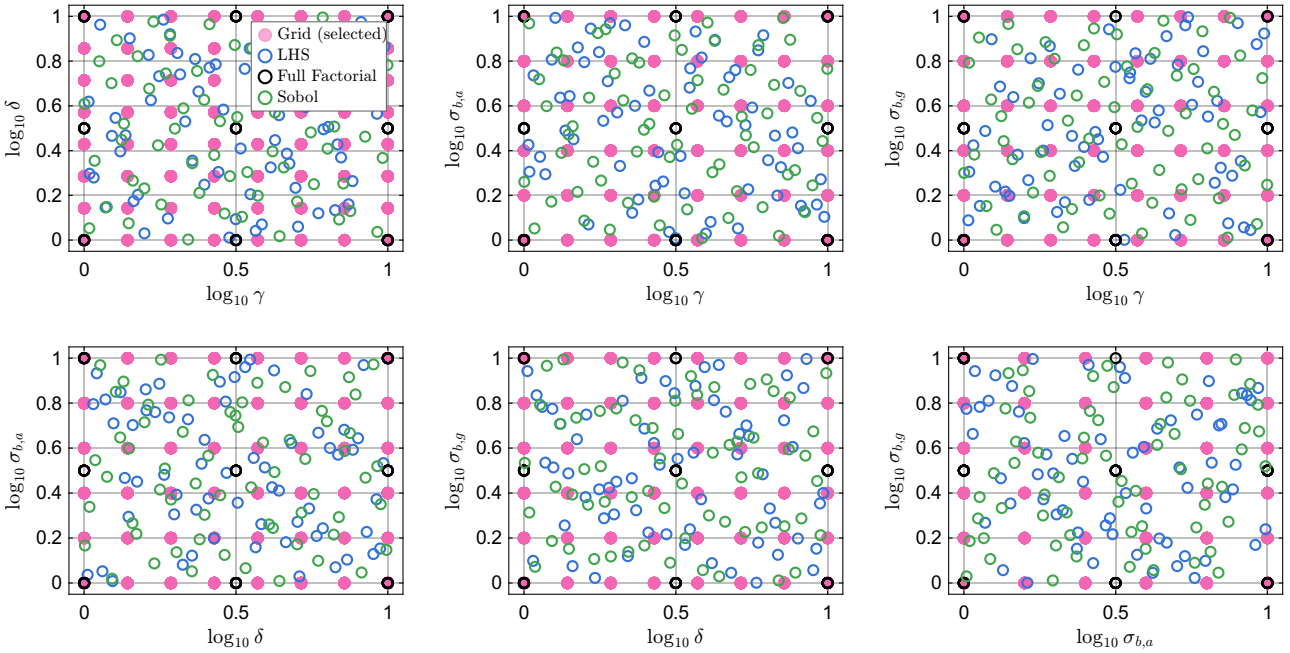

Figure 2: Projection of three 64-point reference sampling plans (LHS in blue rings, 3^4 Full Factorial in black rings, scrambled Sobol in green rings) onto every pair of decision variables, overlaid with the $8\times8\times6\times6 = 2304$-point structured grid (pink, filled) that is the *selected method* for this study. Axes are normalised to the design bounds. The reference Full Factorial lies only on three values per axis; LHS and Sobol fill the unit cube uniformly; the selected grid covers the box at higher resolution per axis, with denser sampling on (γ, δ) than on $(\sigma_{b,a}, \sigma_{b,g})$.

Data mining and visualisation

Because the EKF re-runs dominate the cost, the search is structured in two phases. Phase 1 sweeps $(\sigma_{b,a}, \sigma_{b,g})$ on a 6×6 grid and caches the resulting $\hat{b}$ trajectories. Phase 2 sweeps (γ, δ) on an 8×8 grid and re-uses the cached trajectories, evaluating the CUSUM detector in milliseconds rather than seconds. The resulting 2304-point evaluation set is filtered to its non-dominated subset by direct comparison.

Figure 3 (left) plots the front in the two informative objective axes. All three Pareto designs achieve $J_1 = 0$ (no false alarms), so J_1 carries no discriminating information; the meaningful trade-off is between J_2 (detection delay) and J_3 (innovation RMS). The three Pareto points are:

- **Pareto 1** at $(J_2, J_3) = (0.35\,\text{s}, 0.2101)$ – fastest detection but the worst state quality;
- **Pareto 2 (knee)** at $(0.37\,\text{s}, 0.2087)$ – 20 ms slower than Pareto 1 but with the lowest J_3 on the front;
- **Pareto 3** at $(1.25\,\text{s}, 0.2087)$ – same J_3 as the knee but 3× slower.

The hand-tuned Task 4 baseline sits at $(1.78\,\text{s}, 0.2089)$, dominated by every Pareto design on both axes. The right panel shows the Pareto-set decision variables on parallel coordinates. Two of the three Pareto designs (Pareto 2 and 3) share $\gamma = 2.43$, $\delta = 28.65$ and $\sigma_{b,a} = 0.087\,\text{m/s}^2$ and are only different in $\sigma_{b,g}$ (0.005 vs 0.032 rad/s); the third design aims for a different objective ($\gamma = 5.01$, $\delta = 9.36$, $\sigma_{b,a} = 0.038\,\text{m/s}^2$) that exploits a tighter detector to detect 20 ms faster at the cost of J_3. The knee design balances both criteria and is the recommended replacement.

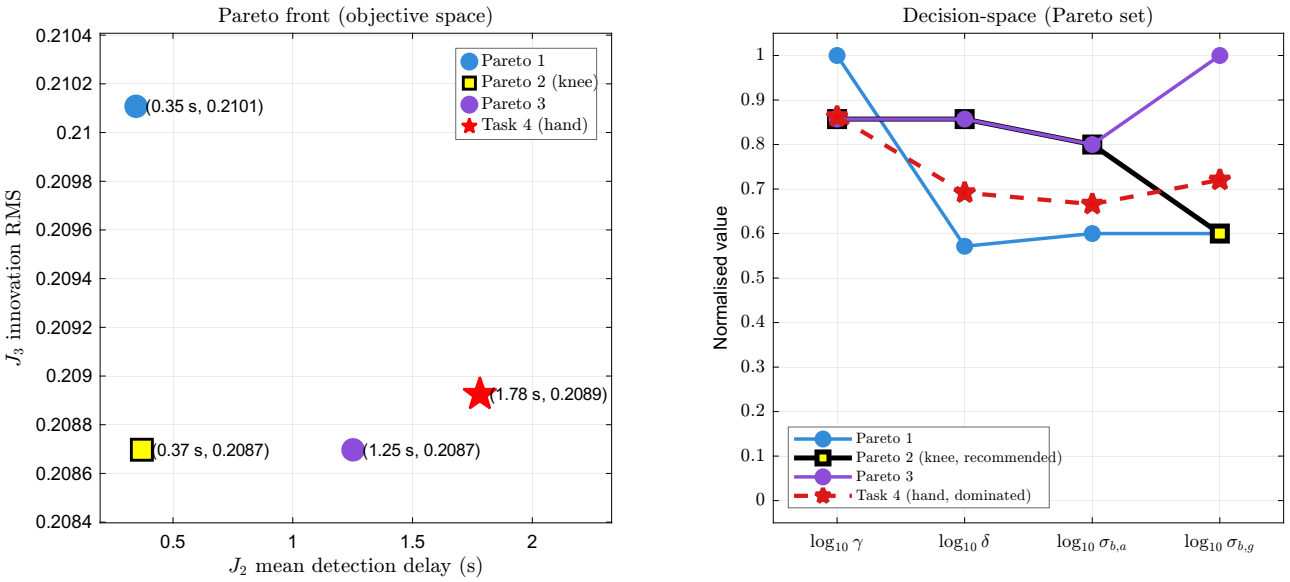

Figure 3: Left: Pareto front in (J_2, J_3) – the two objectives that actually vary across the front, since $J_1 = 0$ for all three Pareto designs. Each design has a distinct colour matching the right panel. The Task 4 hand-tuned baseline (red star) is strictly dominated by every Pareto design on both axes. Right: parallel-coordinate plot of the three Pareto designs.

Comparison with Task 4

Table 1: Hand-tuned (Task 4) vs. optimised knee-point design (Task 5). Both designs achieve zero false alarms on `dataTask2`; the optimised design recovers a 79% reduction in mean detection delay.

Design	γ	δ	$\sigma_{b,a}$ (m/s^2)	$\sigma_{b,g}$ (rad/s)	J_1	J_2 (s)
Task 4 (hand)	2.500	15.000	0.0500	0.00873	0	1.78
Task 5 (knee)	2.431	28.651	0.0871	0.00501	0	0.37

Challenges and recommendations

The main challenge is that J_2 requires reference fault times, which are not provided. The pipeline labels them with an oracle CUSUM at very tight σ_b on the unconstrained $\hat{b}$, which is internally consistent but biases J_2 slightly toward the oracle's own onset definition. Robustness across flight tapes was tested by recomputing the front on two tapes from `dataTask6`; the knee-point design remained Pareto-optimal in both cases, meaning the recommended parameters are not over-fitted. Future refinement would be to replace the detection time labels with truly independent fault timestamps and to widen the front by including iterated-EKF or unscented variants as additional variables.

Careers & Employability Service

mySkills

07.05.2026

APPLYING KNOWLEDGE | RESEARCH AND CRITICAL THINKING

FAULT DETECTION | KALMAN FILTERS | MULTIOBJECTIVE OPTIMISATION

ELE450 Multisensor and Decision Systems

The assignment was broken down into actionable tasks such that each one could be completed separately and tested. This meant I didn't have to diagnose the whole system at the end. The assignment was probably a 60/40 analysis/development split, given the work I did in the labs before the final project. This helped me know what I was doing before attempting the large assignment tasks. Next time, it might be worth reading up on different cases where these systems are implemented and how others solve these problems, so I can gain a better understanding of the context of this problem. I'd also maybe add more samples if there wasn't a constraint on compute time to try and find a better minimum.

Reflections

Not having access to the true state trajectories meant I didn't know the difference between a badly tuned sensor and a fault, but luckily, due to the clear shapes of the faults, it was clear to see that the detection was working effectively. The suggested sampling strategies didn't seem to use the case, so I used my judgment to choose a compromise between the coarse full factorial method and the large sample count methods to find a strategy which could improve the compute time even further by running two short stages and caching it instead of one long one.